

分享主题：大型集群中的数据处理： 如何兼顾效率、可扩展性与可持续性

张晨

京东云 高级工程师
zhangchen.kidd@jd.com

2025/11/15





张晨

嘉宾职位： 京东云 高级工程师

个人简介：

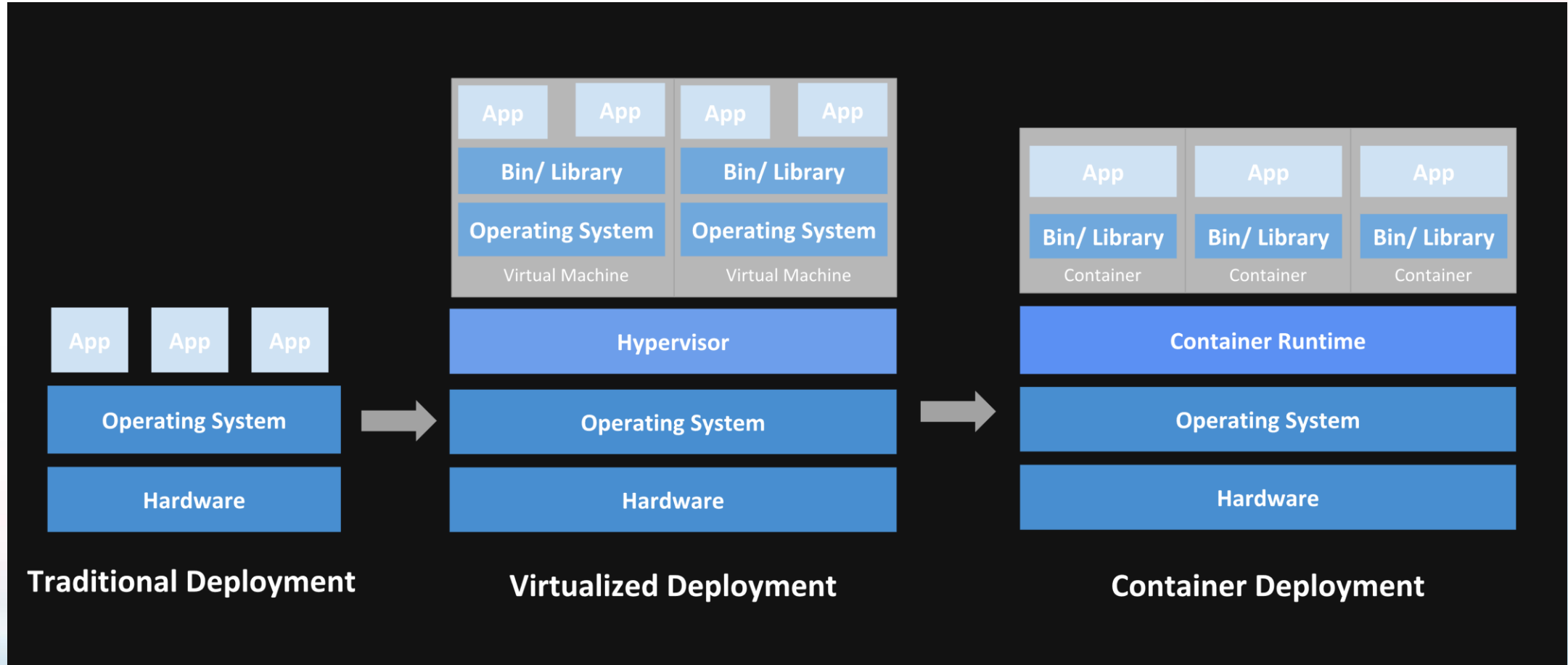
Chen 是位开源软件开发者，主要专注于虚拟化以及通用高可用等基础软件领域。 QEMU社区 maintainer.

章节目录

1. 背景介绍
2. 效率为先
3. 可扩展与可持续

背景

Historical context for Kubernetes



三元悖论



效率为先

Traditional deployment >> Virtualized deployment ---> Security-- Cost-- Efficiency++

Virtualized deployment >> Container deployment ---> Security-- Cost-- Efficiency++

**Why does security always to compromised ?
只有安全受伤的世界已达成**

效率为先

隐形成本 ++

随着基础软件复杂度的提升，越来越少公司可以maintain整个基础软件栈分工越来越细化。

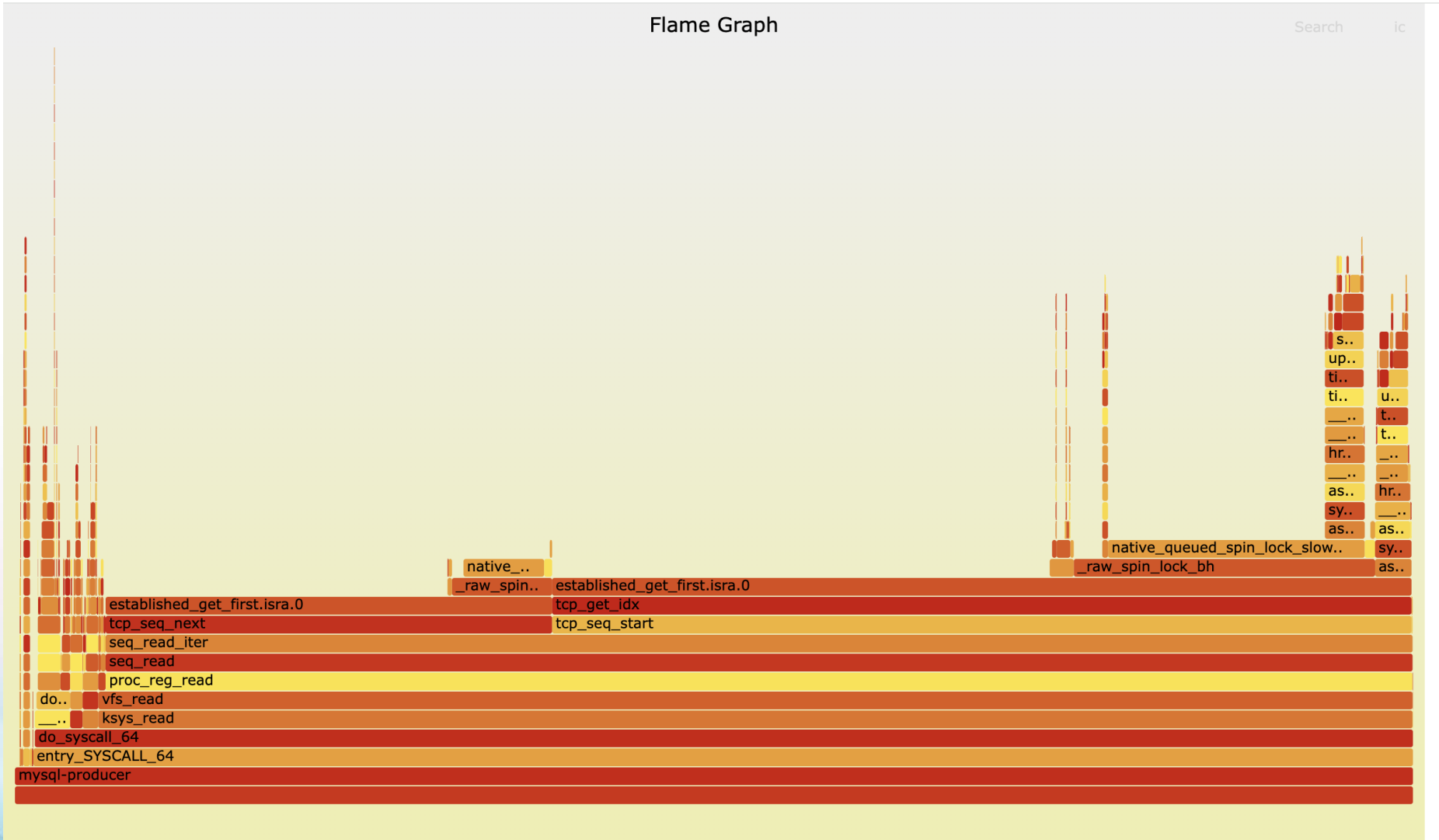
---->厂商大部分的基础设施都选择基于开源项目来降低成本

---->软件维护/部署/二次开发成本提升

---->软件开发者雇佣成本提升

效率为先

现实案例：



效率为先

现实案例：

- “mysql-producer”进程在调用 syscalls, 进入 kernel, 很多时间花在 do_syscall_64 → ksys_read / vfs_read / proc_reg_read → seq_read 的路径上。这说明进程在读用户空间某个文件（很可能是 /proc/net/tcp 或类似 proc 文件）
- 在读 proc 相关内容的时候，有函数 established_get_first.isra.0、tcp_get_idx、seq_read_iter 等在热点里，这些是与 TCP socket 状态集中遍历 / 查看 established sockets 的函数
- 还有锁／自旋锁（_raw_spin_lock_bh / native_queued_spin_lock_slowpath）出现在调用路径里，这说明内核在读这些结构（TCP socket list）的时候，需要获取某些锁，可能竞争较重

所以总的来看，这个进程在读和解析 TCP socket 状态的过程中消耗了比较明显的 CPU / 内核资源，导致了其他container中的服务质量抖动。

效率为先

以当今的技术实现

持续提升效率降低成本会导致安全事故的频繁发生，反向限制了服务质量的提升，导致不可持续。

可扩展与可持续

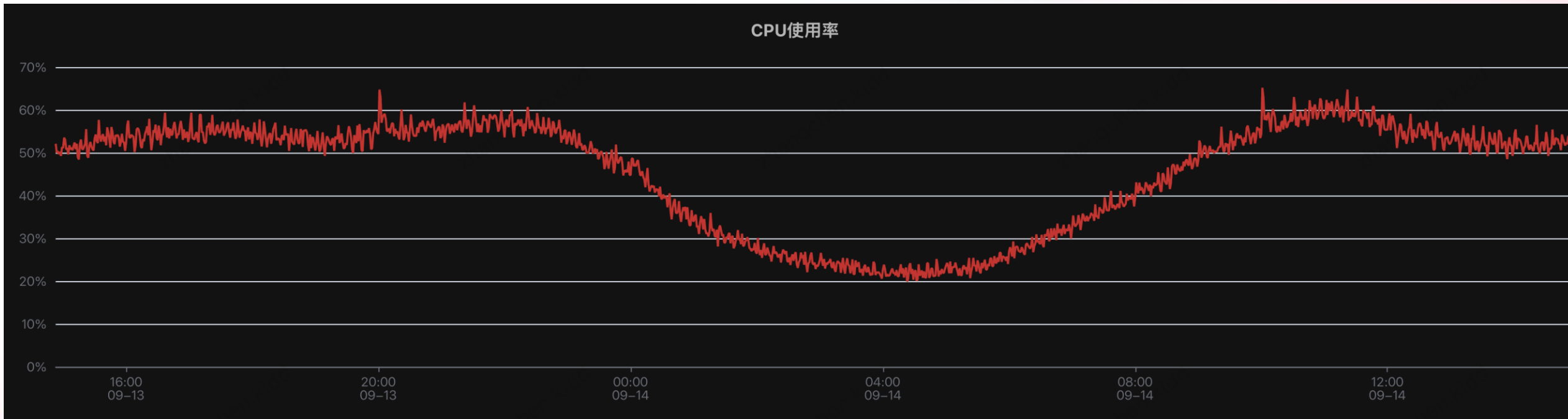
k8s统一调度物理机资源，配合kernel/hypervisor 优化来混合部署kata/runc,

根据实际业务需求配置不同的policy，动态调整 security/efficiency所占比重， 以达到服务质量最大化

作为新技术来临之前的折中方案

可扩展与可持续

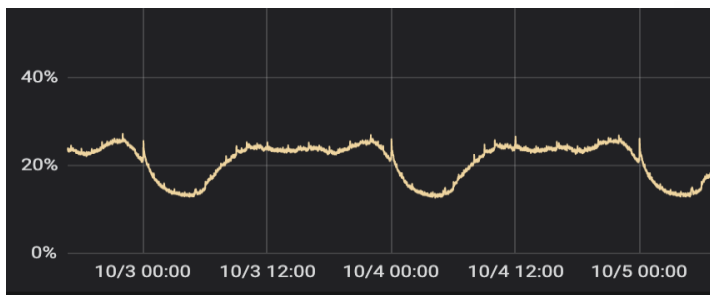
以昼夜业务需求变化为例



可扩展与可持续

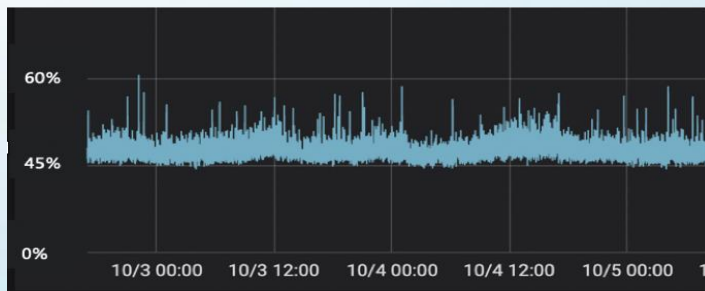
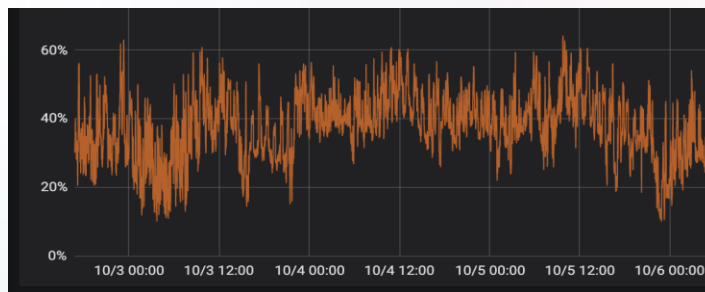
在线业务

有规律的波峰波谷
整体利用率较低



离线业务

任务比较充沛
整体利用率较高



混合部署

没有明显波峰波谷
整体利用率常态运行在较高水平

可扩展与可持续

抗干扰机制



京东容器云全场景混部主要通过下面3种优化措施，来控制
和降低干扰率

1 应用分级

- 通过对在线应用和离线作业按高中低优先级进行分类打标，为调度和资源隔离提供指导信息

2 调度优化

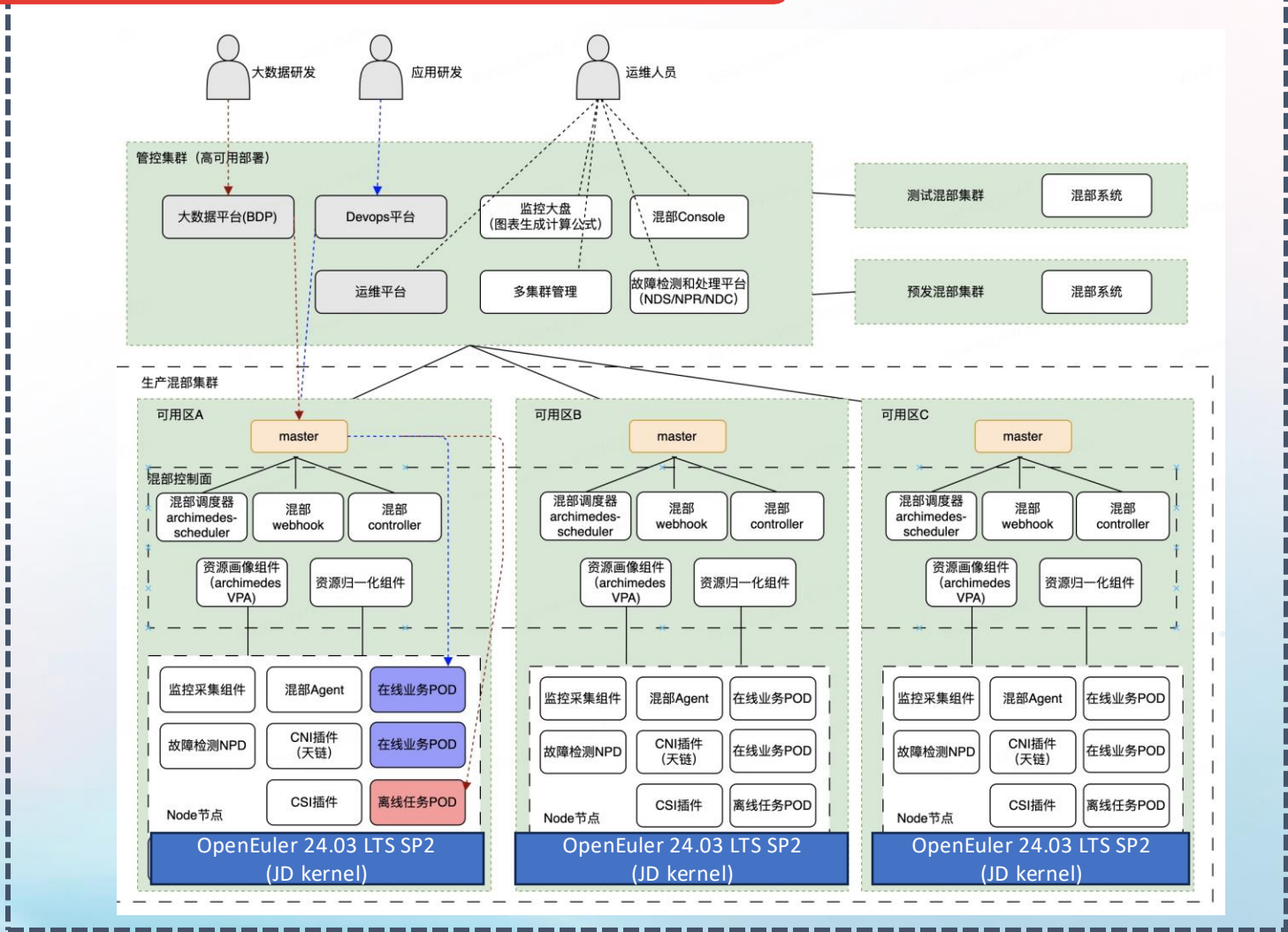
- 优化离线和在线调度速度、对高优先级任务进行绑核，引入离线大框，cpu burst，内存异步回收，磁盘和网络io优先级限速降低tp99影响

3 内核优化

- 优化调度时延，将在线的调度时延降低到1us；优化离线任务的负载均衡策略，减少对运行中的在线任务干扰

可扩展与可持续

京东混部技术特性介绍的部署方案 (以生产区为例)



可扩展与可持续

在线类型 (延时敏感, 中高优先级)

- 零售核心交易系统如库存、商品等
- 中间件如开源缓存数据库, 开源消息队列软件等
- 开源监控索引引擎等
- 搜索、推荐、广告等
- AI 推理

大数据类型 (延时不敏感, 低优先级)

- 实时计算
- 离线大数据
- AI训练

在线级别	名称	定义	SLA要求	QoS	例子
P0	核心系统	黄金交易流程相关系统及其强依赖的基础服务和中间件, 需7*24不间断运行	99.995%	Exclusive/reserved	促销系统、订单系统
P1	次核心系统	系统降级或不可用, 会间接影响用户使用核心功能及用户体验	99.95%	Reserved/shared	开放平台
P2	支撑系统	发生不可用只影响该基础服务自身功能, 不影响0级和1级系统相关业务的连续性	99.9%	shared	商城推荐应用

可扩展与可持续

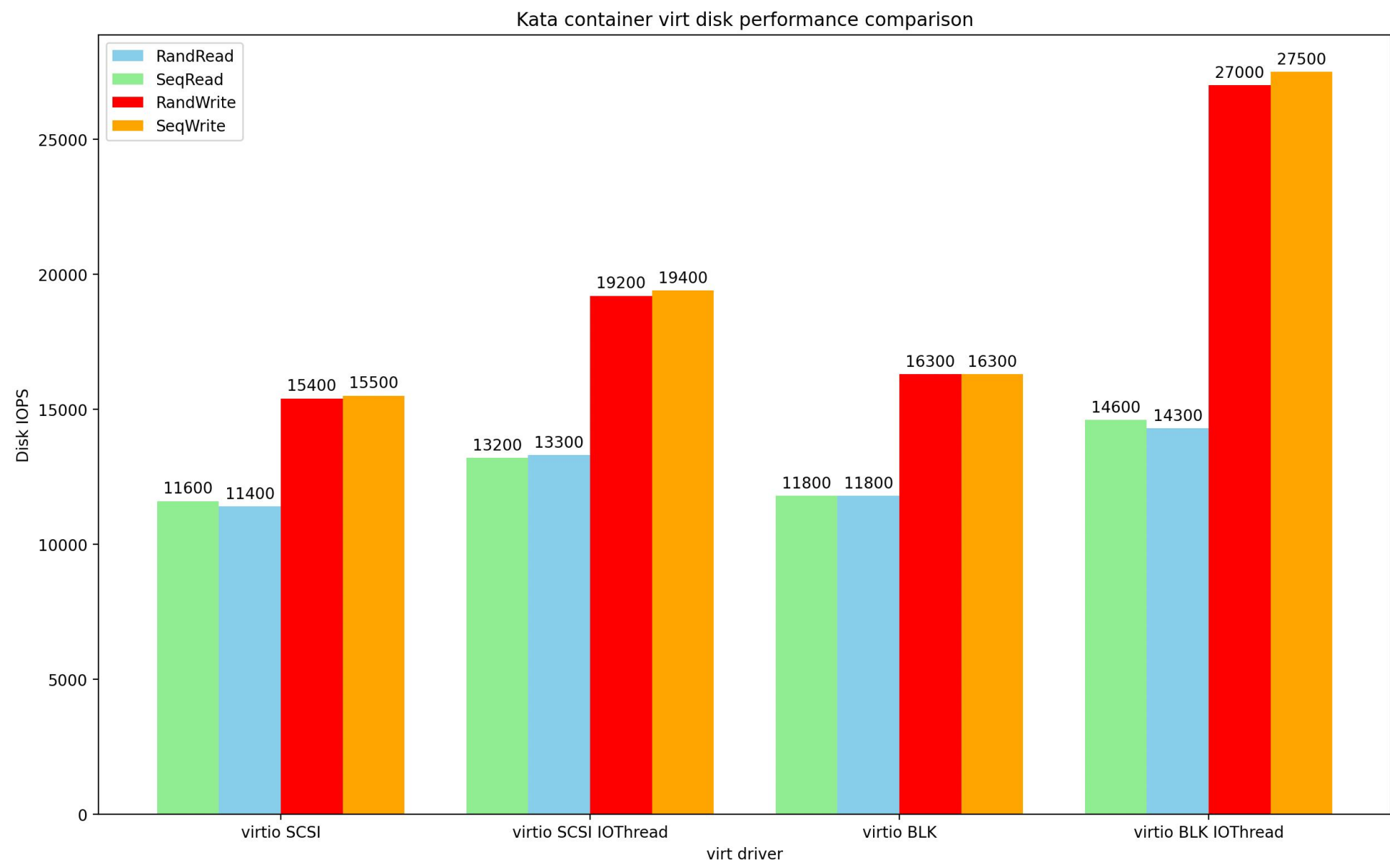
Kata container 基础性能优化

Enable Indep iothreads framework and Introduce virtio-blk with iothreads

What is the indep_iothreads:

This new feature introduce a way to pre-alloc IOThreads for QEMU hypervisor (maybe other hypervisor can support too).Independent IOThreads enables IO to be processed in a separate thread. To generally improve the performance of each module, avoid them running in the QEMU main loop.

可扩展与可持续



可扩展与可持续

Operation	virtio SCSI IOPS	virtio SCSI with iothread IOPS	virtio BLK IOPS	virtio BLK with iothread IOPS	virtio BLK with iothread /SCSI with iothread	virtio BLK with iothread/virtio BLK
RandRead	11400	13300	11800	14300	+8%	+21%
SeqRead	11600	13200	11800	14600	+11%	+24%
RandWrite	15400	19200	16300	27000	+41%	+66%
SeqWrite	15500	19400	16300	27500	+42%	+69%

New feature 已提交Kata 社区并已获得approve

<https://github.com/kata-containers/kata-containers/pull/11620>

可扩展与可持续

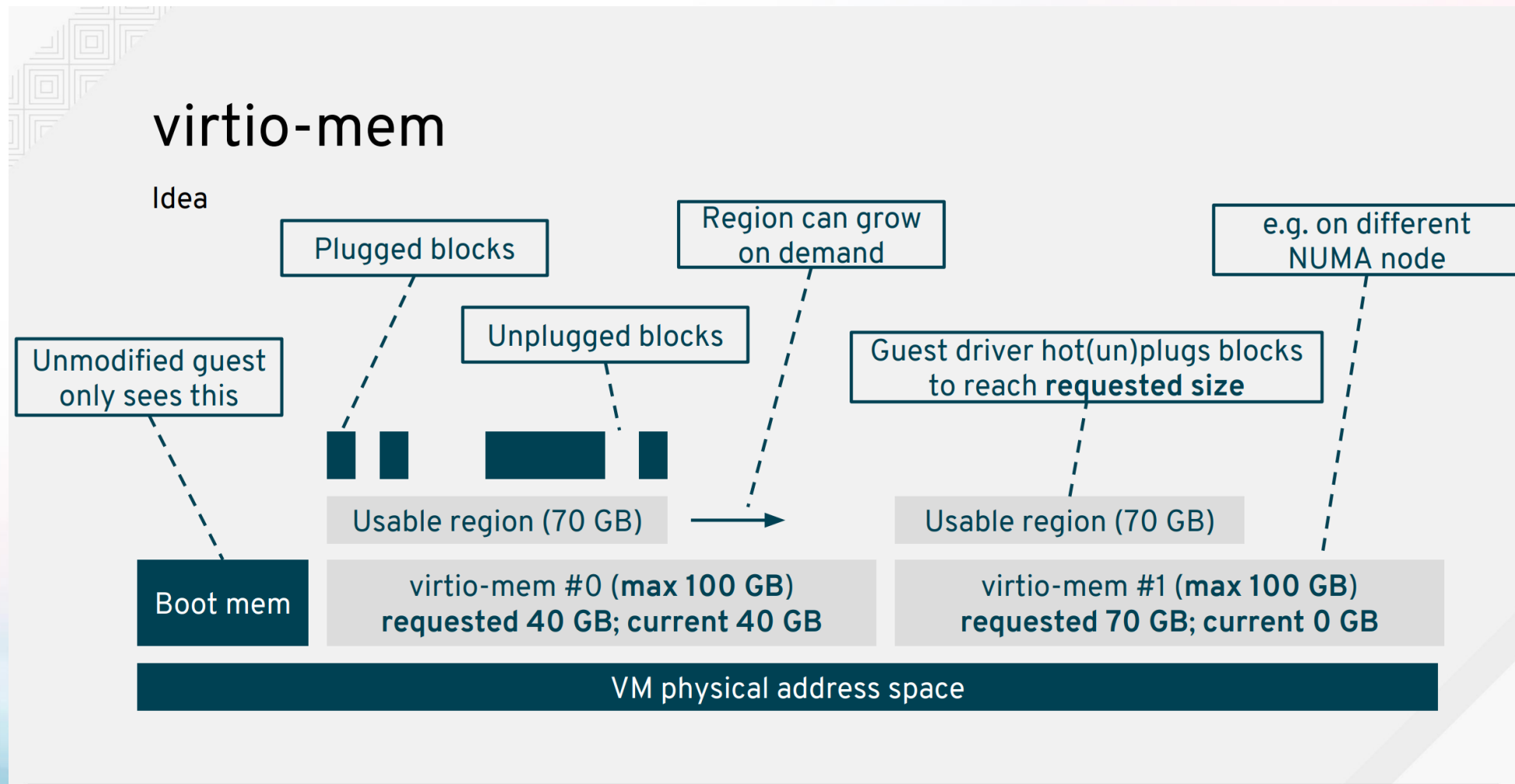
Kata container 基础性能优化

Virtio-mem

The basic idea of virtio-mem is to provide a flexible, cross-architecture memory hot(un)plug solution that avoids many limitations imposed by existing technologies, architectures, and interfaces. More details can be found below and in linked material.

可扩展与可持续

Kata container 基础性能优化



可扩展与可持续

Kata container 基础性能优化

Issues:

X86 machine A:

Virtio memory initial 1T memory VM -----> 45s

X86 machine B:

Virtio memory initial 1T memory VM -----> 230s
(Already synced with CPU manufacturer)

可扩展与可持续

```

1577     → register_memory_blocks_under_node_hotplug(nid, PFN_DOWN(start),
1578     → → → → → → → → → → PFN_UP(start + size - 1));
1579
1580     → /* create new memmap entry */
1581     → if (!strcmp(res->name, "System RAM"))
1582     → → → → firmware_map_add_hotplug(start, start + size, "System RAM");
1583
1584     → /* device_online() will take the lock when calling online_pages() */
1585     → mem_hotplug_done();
1586
1587     → /*
1588     → * In case we're allowed to merge the resource, flag it and trigger
1589     → * merging now that adding succeeded.
1590     → */
1591     → if (mhp_flags & MHP_MERGE_RESOURCE)
1592     → → → → merge_system_ram_resource(res);
1593
1594     → /* online pages if requested */
1595     → if (mhp_get_default_online_type() != MMOP_OFFLINE)
1596     → → → → walk_memory_blocks(start, size, NULL, online_memory_block);
1597
1598     → return ret;
1599 error:
1600     → if (IS_ENABLED(CONFIG_ARCH_KEEP_MEMBLOCK))
1601     → → → → memblock_remove(start, size);
1602 error_mem_hotplug_end:
1603     → mem_hotplug_done();
1604     → return ret;
1605 }

```

可扩展与可持续

```
1459  
1460  —————> /* Use regular block if RAM is smaller than MEM_SIZE_FOR_LARGE_BLOCK */  
1461  —————> if (boot_mem_end < MEM_SIZE_FOR_LARGE_BLOCK) {  
1462  —————> —————> bz = MIN_MEMORY_BLOCK_SIZE;  
1463  —————> —————> goto done;  
1464  —————> }  
1465
```

Guest kernel side:

The default policy need initial memory > 64G

The “MIN_MEMORY_BLOCK_SIZE”(128M) will result in very low efficiency in allocating large amounts of memory.

可扩展与可持续

```
commit 5edba0e6108f692d906977bdb9514d7f6e76d4a7 (HEAD -> for_kata_chen, origin/for_kata_chen)
```

```
Author: zhangchen.kidd <zhangchen.kidd@jd.com>
```

```
Date:   Wed Oct 22 21:47:00 2025 +0800
```

```
arch/x86/mm/init_64.c: Introduce jd_mem_block_size for adjust mem_block
```

```
Because of mini size of memory block size will slow down memory hotplug.
```

```
So, JD will change it in kernel command line.
```

```
Current implementation just support 128M,256M,512M,1G,2G.
```

```
For example: setup kernel cmd to "jd_mem_block_size=2G"
```

```
In JD's use case, it will optimize memory hotplug time for 1T memory:
```

```
Signed-off-by: zhangchen.kidd <zhangchen.kidd@jd.com>
```

Pros :

X86 machine A:

1T memory ~30s ----->>> ~10s (X3 times optimization)

X86 machine B:

1T memory ~230s ----->>> ~10s (X23 times optimization)

Cons :

Memory hotplug block size from 128M ---> 2G (Depends on application requirements)

可扩展与可持续

Summary:

京东云已实现常态化混合部署，经过多次6.18等大促期间服务保障已积累充分经验和数据，混合部署规模已超过百万pods，物理机整机CPU平均利用率已超过45%

Thanks

Any questions -->

